

GTWizard UltraScale+ Transceivers - Example Design, Explained Inch by Inch

[GTWizard UltraScale+ Transceivers — Example Design, Explained Inch by Inch](#)

- [0. What this project actually is](#)
- [1. The file inventory and what each one is for](#)
- [2. The module hierarchy \(instantiation tree\)](#)
- [3. Top-level ports of `example\_top` \(the real hardware boundary\)](#)
- [4. Naming convention used throughout](#)
- [5. Clocking dataflow — inch by inch](#)
 - [5.1 Reference clock path](#)
 - [5.2 Free-running clock path](#)
 - [5.3 TX user clock path \(data path clock, TX side\)](#)
 - [5.4 RX user clock path](#)
 - [5.5 Why TXUSRCLK2 vs TXUSRCLK exist separately](#)
 - [5.6 TX/RX user-clock reset gating \(in `example\_top.v`\)](#)
- [6. Reset dataflow — inch by inch](#)
 - [6.1 What the core's reset controller helper block expects](#)
 - [6.2 Completion feedback](#)
 - [6.3 The `example\_init` bring-up/retry FSM in detail](#)
 - [6.4 Synchronization discipline used throughout resets](#)
- [7. Transmit dataflow — inch by inch](#)
 - [7.1 `prbs\_any\_internals` \(both generate and check modes\)](#)
- [8. Receive dataflow — inch by inch](#)
- [9. Link status / “leaky bucket” dataflow — inch by inch](#)
 - [9.1 The link state machine \(two states, “leaky bucket”\)](#)
 - [9.2 The latched “link ever went down” indicator](#)
 - [9.3 Feedback into the init FSM](#)
- [10. VIO \(hardware debug\) dataflow — inch by inch](#)
- [11. Simulation workflow \(`example\_top\_sim.v` \)](#)
- [12. Hardware bring-up workflow \(putting this on an actual board\)](#)
- [13. Summary of the complete dataflow \(single diagram in words\)](#)
- [14. A few subtleties worth flagging explicitly](#)

GTWizard UltraScale+ Transceivers — Example Design, Explained Inch by Inch

0. What this project actually is

This is the **AMD/Xilinx UltraScale Transceivers Wizard IP (`gtwizard_ultrascale_0`) example design**, auto-generated by Vivado when you customize the GTWizard IP core and tell it to produce an example design.

It is **not** the transceiver core itself — the actual `GTHE4_CHANNEL` / `GTHE4_COMMON` primitives and their wrapper (`gtwizard_ultrascale_0` , referenced but not included in your upload) are generated separately as the “core.” What you uploaded are the files that surround that core: a demonstration/test harness that

1. brings up **one GTH transceiver channel** (`ch0`),
2. drives it with a **known pattern (PRBS31)**,
3. **loops the pattern back** (in simulation) or connects it to a real serial link (in hardware),
4. **checks** the received pattern against what was expected,
5. manages **reset sequencing** so the transceiver comes up correctly,
6. exposes a small **VIO (Virtual I/O)** debug interface so you can watch/poke signals from Vivado Hardware Manager while the FPGA is running.

Configuration clues from the code: - Only **one channel** is enabled (`P_CHANNEL_ENABLE` has a single `1` bit) -> single-lane design. - One reference clock: `MGTREFCLK0_X0Y2` (100/125-ish MHz differential pair via `IBUFDS_GTE4`). - User data width is **32 bits** (`gtwiz_userdata_tx_in` / `_rx_out` are `[31:0]`). - The TX and RX helper-block “user clocking” logic is **excluded from**

the **core** and instead delivered as separate example-design modules (`example_gtwiz_userclk_tx.v` / `_rx.v`) — meaning you chose “not core-contained” for that helper block during IP customization. - The design uses **raw/no-encoding data** (“stimulus_raw” / “checking_raw” — no 8b/10b or 64b/66b framing, just a PRBS pattern straight into `gtwiz_userdata_tx_in`).

1. The file inventory and what each one is for

File	Role
<code>gtwizard_ultrascale_0_example_top.v</code>	Top-level example design. Instantiates buffers, PRBS stimulus/checker, link-status state machine, init state machine, VIO, and the wrapper. This is what you’d actually put in your FPGA project (or feed to synthesis).
<code>gtwizard_ultrascale_0_example_top_sim.v</code>	Simulation-only testbench. Instantiates <code>example_top</code> , generates clocks, loops TX serial data back to RX serial input, and self-checks pass/fail. Never goes to hardware.
<code>gtwizard_ultrascale_0_example_wrapper.v</code>	Instantiates the actual IP core (<code>gtwizard_ultrascale_0</code>) plus any “helper blocks” you excluded from the core (here: TX/RX user clocking networks). Wires the two together and forwards the rest of the ports up to <code>example_top</code> .
<code>gtwizard_ultrascale_0_example_wrapper_functions.v</code>	A <code>`include</code> file of pure Verilog functions used only at elaboration time to compute bit-vector indices (which slice of a big packed bus belongs to which channel/common). Needed because Vivado generates this file to scale automatically for designs with many channels — even though here there’s only one.
<code>gtwizard_ultrascale_0_example_gtwiz_userclk_tx.v</code>	TX user clocking helper block. Takes <code>TXOUTCLK</code> from the transceiver and derives <code>TXUSRCLK</code> / <code>TXUSRCLK2</code> using <code>BUFG_GT</code> clock buffers/dividers, plus produces an “active” status flag.
<code>gtwizard_ultrascale_0_example_gtwiz_userclk_rx.v</code>	Same as above, but for the receive side (<code>RXOUTCLK</code> -> <code>RXUSRCLK</code> / <code>RXUSRCLK2</code>).
<code>gtwizard_ultrascale_0_example_init.v</code>	Initialization/retry state machine. Watches TX/RX reset-done and “data good” signals, with timeout timers, and re-triggers resets if bring-up isn’t completing (handles flaky reference clocks/links at power-up).
<code>gtwizard_ultrascale_0_example_reset_sync.v</code>	Generic async-assert / sync-deassert reset synchronizer (5-FF chain) used everywhere resets cross clock domains.
<code>gtwizard_ultrascale_0_example_bit_sync.v</code>	Generic 2-state bit synchronizer (5-FF chain, no async reset) used to bring single-bit status signals safely into another clock domain (e.g., for the VIO).
<code>gtwizard_ultrascale_0_example_stimulus_raw.v</code>	TX pattern generator. Wraps <code>prbs_any</code> in generate mode to produce a PRBS31 stream on <code>txdata_out[31:0]</code> , synchronized to <code>TXUSRCLK2</code> .
<code>gtwizard_ultrascale_0_example_checking_raw.v</code>	RX pattern checker. Wraps <code>prbs_any</code> in check mode; compares <code>rxdata_in[31:0]</code> against the expected PRBS31 sequence and outputs a per-cycle <code>prbs_match_out</code> .
<code>gtwizard_ultrascale_0_prbs_any.v</code> (a.k.a. <code>PRBS_ANY.v</code> , from XAPP884)	The actual parameterizable PRBS LFSR core used by both the stimulus and checking modules — one generic module, instantiated twice with different parameters (<code>CHK_MODE=0</code> for generator, <code>CHK_MODE=1</code> for checker).
<code>gtwizard_ultrascale_0_example_top.xdc</code>	Constraints file: pin locations for the reference clock, clock period constraints (<code>create_clock</code>), false-path constraints on synchronizer registers, and CDC (clock-domain-crossing) waivers for Vivado’s CDC checker so it doesn’t flag the intentional synchronizer paths as bugs.

Not included in your upload, but referenced/required to actually build this: `gtwizard_ultrascale_0.v` (the actual GT core wrapper around `GTHE4_CHANNEL` / `GTHE4_COMMON`), `gtwizard_ultrascale_0_vio_0` (a Vivado VIO IP instance), and the core-level XDC.

2. The module hierarchy (instantiation tree)

<code>gtwizard_ultrascale_0_example_top_sim</code> (simulation only)
--

```

+- gtwizard_ultrascale_0_example_top (real top level – synthesizable)
+- IBUF (buffers hb_gt wiz_reset_all_in)
+- BUFG (buffers the free-running clock)
+- IBUFDS_GTE4 (differential ref-clock buffer)
+- example_stimulus_raw (TX PRBS generator)
| +- example_reset_synchronizer
| +- prbs_any (CHK_MODE=0, generate mode)
+- example_checking_raw (RX PRBS checker)
| +- example_reset_synchronizer
| +- prbs_any (CHK_MODE=1, check mode)
+- example_reset_synchronizer (for PRBS-mismatch-to-freerun-domain sync)
+- example_bit_synchronizer (link_down_latched_reset sync)
+- example_init (bring-up / retry FSM)
| +- example_reset_synchronizer (reset_all sync)
| +- example_bit_synchronizer x2 (tx_init_done, rx_init_done)
| +- example_bit_synchronizer (rx_data_good)
+- example_bit_synchronizer x5 (VIO status synchronizers: gtpowergood,
| txpmaresetdone, rxpmaresetdone,
| gt wiz_reset_tx_done, gt wiz_reset_rx_done)
+- gtwizard_ultrascale_0_vio_0 (Vivado VIO debug core)
+- example_wrapper
| +- example_gt wiz_userclk_tx (TX user clock network)
| | +- BUFG_GT x1 or x2
| +- example_gt wiz_userclk_rx (RX user clock network)
| | +- BUFG_GT x1 or x2
| +- gtwizard_ultrascale_0 (the actual IP core – NOT in your upload)

```

3. Top-level ports of `example_top` (the real hardware boundary)

```

input mgtrefclk0_x0y2_p / _n // differential GT reference clock (100 MHz class)
input ch0_gthrxn_in / ch0_gthrxp_in // serial RX pair for channel 0
output ch0_gthtxn_out / ch0_gthtxp_out // serial TX pair for channel 0
input hb_gt wiz_reset_clk_freerun_in // free-running clock (drives all reset/init/VIO logic)
input hb_gt wiz_reset_all_in // master "reset everything" button
input link_down_latched_reset_in // user-driven clear for the "link ever went down" flag
output link_status_out // 1 = link currently up (PRBS matching)
output link_down_latched_out // sticky: 1 once link has ever gone down since last clear

```

That’s it — everything else (32-bit user data, per-channel clock, per-channel reset controls) is internal, generated/consumed by the example design’s own stimulus/checker logic rather than exposed off-chip. This is intentional: it’s a **self-contained, self-testing loopback design**.

4. Naming convention used throughout

The comments in `example_top.v` spell this out explicitly: - `hb#...` — signals connected to a **helper block** (reset controller, user clocking, etc.), `#` = instance number. - `cm#...` — signals connected to a **transceiver common primitive** (shared PLL block), `#` = common instance number. - `ch#...` — signals connected to a **transceiver channel primitive**, `#` = channel number. - `..._int` — internal wire, purely for connecting one block’s output to another block’s input inside `example_top`.

Because the IP core internally uses **packed vectors** across all possible channels/commons (to scale to multi-channel designs), `example_top` “de-vectorizes” everything down to per-channel/per-common wires (even though here there’s only one of each), using simple 1-bit slice assigns like:

```

wire [0:0] gthrxn_int;
assign gthrxn_int[0:0] = ch0_gthrxn_in;

```

This is boilerplate that Vivado generates automatically and scales up for N-channel designs; for your single-channel case it’s essentially a pass-through renaming layer.

5. Clocking dataflow — inch by inch

5.1 Reference clock path

```

mgtrefclk0_x0y2_p/n (board pins)
-> IBUFDS_GTE4 (differential->single-ended GT buffer, REFCLK_EN_TX_PATH=0 i.e. RX-only ref path use here)
-> mgtrefclk0_x0y2_int
-> cm0_gtrefclk01_int -> gtrefclk01_int -> wrapper.gtrefclk01_in
-> core (feeds the QPLL/CPLL inside GTHE4_COMMON)

```

The core's PLL (QPLL1 here, based on `qpll1outclk_out` / `qpll1outrefclk_out` ports) multiplies this reference up to the line rate and also outputs `qpll1outclk` / `qpll1outrefclk`, which in a multi-quad design would feed neighboring quads; here they're just brought out to the wrapper boundary and left unconnected further up (single-quad design).

5.2 Free-running clock path

```
hb_gt wiz_reset_clk_freerun_in (external clock source, independent of the GT PLLs)
-> BUFG (global clock buffer)
-> hb_gt wiz_reset_clk_freerun_buf_int
```

This buffered free-running clock is the **backbone clock domain** for almost everything that isn't raw serial data: the init state machine, the link-status state machine, the VIO, and every synchronizer's destination domain. It must be running and stable *before* the GT reference clock/PLLs even lock, which is why it's a separate, independent clock input — its whole purpose is to drive reset/control logic even while the transceiver itself is not yet up.

The XDC pins this at a 7.529 ns period (~132.8 MHz), which matches the `P_FREERUN_FREQUENCY = 132.8125` parameter default passed into `example_init`.

5.3 TX user clock path (data path clock, TX side)

```
core.txoutclk_out (TXOUTCLK of the master TX channel – line-rate-derived clock)
-> wrapper: txoutclk_int
-> gt wiz_userclk_tx_srcclk_out (= txoutclk_int[master_channel_index])
-> example_gt wiz_userclk_tx module:
    BUFG_GT (divide by P_USRCLK_DIV) -> gt wiz_userclk_tx_usrclk_out (TXUSRCLK)
    BUFG_GT (divide by P_USRCLK2_DIV, or same buffer if ratio=1) -> gt wiz_userclk_tx_usrclk2_out (TXUSRCLK2)
-> fed back into wrapper as txusrclk_int / txusrclk2_int
-> back into the core's txusrclk_in / txusrclk2_in ports (clocks the TX datapath FIFOs/logic inside the GT)
```

Both `BUFG_GT` instances are held in reset (`.CLR`) by `gt wiz_userclk_tx_reset_in` until told otherwise (see reset flow below). Once released, a 2-FF synchronizer inside `example_gt wiz_userclk_tx` produces `gt wiz_userclk_tx_active_out`, a status bit meaning “TXUSRCLK2 is now toggling and safe to use,” which is what `example_top` uses to gate the PRBS generator and stimulus reset.

5.4 RX user clock path

Identical structure/purpose to 5.3, mirrored for RX: `RXOUTCLK` -> `example_gt wiz_userclk_rx` -> `RXUSRCLK` / `RXUSRCLK2` -> back into the core, with `gt wiz_userclk_rx_active_out` gating the checker.

5.5 Why TXUSRCLK2 vs TXUSRCLK exist separately

`TXUSRCLK` clocks the transceiver's internal PCS logic at the “full” internal parallel rate; `TXUSRCLK2` clocks the user-facing 32-bit data interface, potentially at an integer-divided rate (`P_FREQ_RATIO_USRCLK_TO_USRCLK2`). In this design they may be tied to the same `BUFG_GT` output (ratio = 1) or separately divided — determined at IP-customization time and baked into the instantiation parameters.

5.6 TX/RX user-clock reset gating (in `example_top.v`)

```
assign hb0_gt wiz_userclk_tx_reset_int = ~(&txpmaresetdone_int);
assign hb0_gt wiz_userclk_rx_reset_int = ~(&rxpmaresetdone_int);
```

Translation: hold the TX (RX) clocking network in reset until `TXPMARESETDONE` (`RXPMARESETDONE`) from the transceiver's analog front end (PMA) indicates the underlying serializer/deserializer clock source is stable. Only then release the `BUFG_GT` clock dividers.

6. Reset dataflow — inch by inch

This design layers **three sources** of “reset all” together:

```
assign hb_gt wiz_reset_all_int =
    hb_gt wiz_reset_all_buf_int      // from the external pin, via IBUF
|| hb_gt wiz_reset_all_init_int    // from example_init, when a retry is needed
|| hb_gt wiz_reset_all_vio_int;    // from the VIO, for manual debug-triggered resets
```

Similarly, the RX-only datapath reset ORs two sources:

```
assign hb_gt wiz_reset_rx_datapath_int =
```

```
hb_gtwiz_reset_rx_datapath_init_int // from example_init, RX-only retry
|| hb_gtwiz_reset_rx_datapath_vio_int; // from the VIO
```

6.1 What the core's reset controller helper block expects

The core (not shown, but standard for GTWizard) contains a **reset controller helper block** that sequences: PLL reset -> wait for PLL lock -> TX/RX PMA reset -> wait for TXPMARESETDONE / RXPMARESETDONE -> release TX/RX datapath reset -> assert `gtwiz_reset_tx_done_out` / `gtwiz_reset_rx_done_out` when complete. `example_top` / `example_wrapper` merely feed it the right trigger inputs: - `gtwiz_reset_all_in` — reset absolutely everything (PLL + TX + RX). - `gtwiz_reset_tx_pll_and_datapath_in` / `gtwiz_reset_tx_datapath_in` — TX-only resets (from VIO in this design). - `gtwiz_reset_rx_pll_and_datapath_in` / `gtwiz_reset_rx_datapath_in` — RX-only resets (tied to `hb_gtwiz_reset_rx_pll_and_datapath_int` / `hb_gtwiz_reset_rx_datapath_int`).

Because this design shares one PLL type between TX and RX (`hb_gtwiz_reset_rx_pll_and_datapath_int` is hard-wired to `1'b0`), the “reset RX PLL and datapath” (which would also disturb TX, since they'd share a PLL) is never used — only the RX-datapath-only reset is exercised, so a receiver reset can't take down an already-good transmitter.

6.2 Completion feedback

`gtwiz_reset_tx_done_int` / `gtwiz_reset_rx_done_int` come back out of the core (via the wrapper) once the reset controller's TX/RX sequencing finishes. These feed: - `example_init` (`tx_init_done_in`, `rx_init_done_in`) — used to time out/retry if reset doesn't complete. - `example_checking_raw`'s own reset input (`~hb0_gtwiz_reset_rx_done_int` forces the checker to stay in reset until RX is actually done). - The VIO (synchronized first) for hardware debug visibility.

6.3 The `example_init` bring-up/retry FSM in detail

Four states, all clocked on the free-running clock:

1. **ST_START** — clears the shared timer and reset outputs, moves to `ST_TX_WAIT`.
2. **ST_TX_WAIT** — starts the timer. If `tx_init_done_sync` (synchronized `gtwiz_reset_tx_done`) arrives before the timer hits its **TX terminal count** (`P_TX_TIMER_DURATION_US = 30000 µs = 30 ms` by default), move on to `ST_RX_WAIT`. If the timer expires first, assert `reset_all_out` (fires a full `hb_gtwiz_reset_all_int` reset), bump the retry counter, and go back to `ST_START`. This handles cases like “reference clock wasn't stable yet” causing the TX PLL to never lock.
3. **ST_RX_WAIT** — starts the timer again with a longer budget (`P_RX_TIMER_DURATION_US = 130000 µs = 130 ms` default — long enough to allow for CDR lock and possibly multiple elastic-buffer bypass attempts). When the timer expires, check both `rx_init_done_sync` and `rx_data_good_sync` (the latter driven by the PRBS checker's link-up indicator, described in §7). If both are good, declare `init_done_out = 1` and move to `ST_MONITOR`. If either is bad, assert `reset_rx_out` (an RX-only reset, not a full reset — so a stable TX link isn't disturbed), bump retry counter, go back to `ST_START`.
4. **ST_MONITOR** — steady-state. Continuously watches `rx_init_done_sync` and `rx_data_good_sync`; if either drops (i.e., link is lost or RX resets unexpectedly), clear `init_done_out`, assert `reset_rx_out`, bump the retry counter, and restart the whole sequence from `ST_START`. This is explicitly called out in the top-level comments as something that “can have the effect of overriding or disturbing user-provided inputs” — i.e., this auto-recovery behavior is a demo convenience, not necessarily what you want in a production design.

The `retry_ctr_out[3:0]` register (saturating at 15, never itself reset) is exposed to the VIO purely as a bring-up diagnostic: “how many times has this thing had to retry since programming?”

6.4 Synchronization discipline used throughout resets

Two generic, reusable primitives handle every clock-domain crossing in this design:

- **example_bit_synchronizer** — a plain 5-flip-flop chain (`ASYNC_REG=TRUE` on the first 4 for timing-tool metastability awareness), no reset logic, used for status bits going *into* the free-running clock domain (e.g., `gtpowergood`, `txpmaresetdone`, PRBS “match” status bits used for VIO display).
- **example_reset_synchronizer** — a 5-flip-flop chain that is **asynchronously set** (`posedge rst_in` in the sensitivity list) but **synchronously cleared**, guaranteeing the receiving domain sees the reset assert immediately (no metastability window on assertion, since it's asynchronous) but only releases reset after several clean clock edges (avoiding metastability on deassertion). This is the standard “assert async / deassert sync” reset-synchronizer pattern and is reused everywhere a reset needs to cross into a new clock domain (into TXUSRCLK2 domain for the stimulus generator, into RXUSRCLK2 domain for the checker, into the free-running domain for `reset_all`, etc.).

The XDC file backs this up with `set_false_path` constraints on the metastability flip-flops of both synchronizer types (since by design their first-stage input is asynchronous relative to the destination clock, static timing analysis on that specific path is meaningless) and a long list of `create waiver` commands suppressing Vivado's automatic CDC (clock-domain-crossing) rule checker on these same intentional, properly-synchronized crossings.

7. Transmit dataflow — inch by inch

1. `example_top` instantiates `example_stimulus_inst0` (`example_stimulus_raw`), driven by `TXUSRCLK2` and gated by `gtwiz_userclk_tx_active_in`.
2. Inside `example_stimulus_raw`:
 - A local reset is built: `gtwiz_reset_all_in || ~gtwiz_userclk_tx_active_in` — i.e., stay in reset if either a master reset is asserted, or the TX user clock isn't confirmed active yet.
 - That reset is passed through an `example_reset_synchronizer` into the `TXUSRCLK2` domain.
 - A `prbs_any` instance is configured as a **generator**:

```
.CHK_MODE(0), .INV_PATTERN(1), .POLY_LENGTH(31), .POLY_TAP(28), .NBITS(32)
```

This is a standard **PRBS31** polynomial ($x^{31} + x^{28} + 1$), inverted output, generating 32 new pattern bits every `TXUSRCLK2` cycle. `EN` is tied permanently high — the generator never pauses, ideal for a raw/no-encoding continuous-stream test.
3. `txdata_out[31:0]` from the generator is wired all the way up to `hb0_gtwiz_userdata_tx_int`, then to `gtwiz_userdata_tx_int`, then into the core's `gtwiz_userdata_tx_in[31:0]` port.
4. Inside the (not-included) core, this 32-bit parallel word is serialized by the GTH transceiver's internal SerDes and driven out the differential `GTHTXP / GTHTXN` pins at the line rate set by the PLL, ultimately reaching `ch0_gthtxp_out / ch0_gthtxn_out`.

7.1 prbs_any internals (both generate and check modes)

- `prbs_reg[1:POLY_LENGTH]` is the LFSR state, initialized to all-1s (so it's self-starting without needing an explicit reset for power-on — the module comment notes reset isn't strictly required for that reason, though the design still resets it on link resets).
- A generate for loop unrolls `NBITS` (32) parallel LFSR advances per clock, computing:
 - `prbs_xor_a[I] = prbs[I][POLY_TAP] ^ prbs[I][POLY_LENGTH]` — the LFSR feedback tap XOR.
 - In **generate mode** (`CHK_MODE=0`), the new bit fed back into the shift register is `prbs_xor_a[I]` (i.e., pure LFSR sequence generation) and `prbs_xor_b[I] = prbs_xor_a[I] ^ data_in_i[I]` is what's actually output — this XOR with `DATA_IN` allows the caller to inject bit errors for test purposes (unused here; `DATA_IN` is tied to `32'b0` in `example_stimulus_raw`, so `DATA_OUT` = pure PRBS pattern).
 - In **check mode** (`CHK_MODE=1`), the bit fed back into the shift register comes from the *incoming data* itself (`data_in_i[I]`, i.e., self-synchronizing checker), and the output `prbs_xor_b[I]` becomes an **error flag per bit**: 0 if that bit matched the locally regenerated expected pattern, 1 if it mismatched.
- `INV_PATTERN=1` on both sides inverts the data going in (`data_in_i = ~DATA_IN`), which is the standard trick to avoid a stuck-at-all-zeros PRBS state matching a dead/all-zero line.

8. Receive dataflow — inch by inch

1. Serial data arrives at `ch0_gthrxp_in / ch0_gthrxn_in`, is deserialized by the (uninstantiated-here) GTH channel, clock-recovered via CDR against the incoming data stream, and delivered as a 32-bit parallel word on `gtwiz_userdata_rx_out[31:0]` from the core, clocked by `RXUSRCLK2`.
2. This flows: core -> wrapper (`gtwiz_userdata_rx_out`) -> `example_top` (`gtwiz_userdata_rx_int` -> `hb0_gtwiz_userdata_rx_int`) -> into `example_checking_inst0` as `rxdata_in`.
3. Inside `example_checking_raw`:
 - Local reset = `gtwiz_reset_all_in || ~gtwiz_userclk_rx_active_in`, synchronized into the `RXUSRCLK2` domain.
 - Additionally, at the `example_top` level, the checker's reset input is further OR'd with `~hb0_gtwiz_reset_rx_done_int` — so the checker is *also* forcibly held in reset until the core reports RX reset sequencing complete.
 - `prbs_any` instantiated with `CHK_MODE=1`, same PRBS31/inverted parameters as the generator, so it locally regenerates the same expected sequence and self-synchronizes to the incoming stream.
 - `prbs_any_chk_error_int[31:0]` — one error bit per received data bit, this cycle.
 - A registered NOR reduction: `prbs_match_out <= ~(|prbs_any_chk_error_int)` — i.e., `prbs_match_out` is 1 only if **all 32 bits matched** this cycle, else 0. This is registered (one-cycle latency) so it's glitch-free for downstream logic.

9. Link status / “leaky bucket” dataflow — inch by inch

`prbs_match_out` from the (single, in this design) channel checker feeds into a vector `prbs_match_int[0:0]` in `example_top`. Then:

```
wire prbs_error_any_async = ~(&prbs_match_int); // NAND across all channels: 1 = any channel currently mismatching
```

(With one channel, this is simply the inverse of that channel's match bit — but the code is written generically for any channel count.)

This asynchronous (with respect to the free-running clock) error indicator is passed through an `example_reset_synchronizer` into the free-running clock domain, producing `prbs_error_any_sync`.

9.1 The link state machine (two states, “leaky bucket”)

```
localparam ST_LINK_DOWN = 1'b0, ST_LINK_UP = 1'b1;
reg sm_link = ST_LINK_DOWN;
reg [6:0] link_ctr = 0; // 7-bit counter, range 0-127
```

- **ST_LINK_DOWN**: counts *up* by 1 on every free-running clock cycle where there is currently no PRBS error, but the counter is **reset to 0 the instant any error occurs**. Once the counter reaches 67 consecutive clean cycles, transition to `ST_LINK_UP`. (So: link is only declared “up” after 67 back-to-back all-clean cycles — a strict, noise-intolerant bar to *initially* claim link.)
- **ST_LINK_UP**: once up, the link is more tolerant of occasional bit errors — each clean cycle only adds +1 to the counter (saturating at 67), but each error cycle *subtracts 34* from the counter. If the counter would go at or below the point where subtracting 34 crosses zero (checked via `link_ctr > 33` vs. not), the state drops back to `ST_LINK_DOWN`. This “gain up slowly, lose fast” asymmetric behavior is the classic leaky-bucket approach to link-quality debouncing: a handful of scattered bit errors won't instantly tear the link down, but a burst of errors will.

`link_status_out` is directly assigned to `sm_link` — a real-time “is the link currently considered good” indicator.

9.2 The latched “link ever went down” indicator

```
always @(posedge hb_gtwiz_reset_clk_freerun_buf_int) begin
  if (link_down_latched_reset_sync)
    link_down_latched_out <= 1'b0;
  else if (!sm_link)
    link_down_latched_out <= 1'b1;
end
```

This is a **sticky fault flag**, initialized to `1'b1` at reset (output reg `link_down_latched_out = 1'b1;` in the port declaration) — meaning it starts “true” (link presumed down) until first cleared. It latches high the instant `sm_link` is ever `ST_LINK_DOWN`, and can only be cleared by a synchronized reset input that is itself the OR of the external `link_down_latched_reset_in` pin and a VIO-driven bit (`link_down_latched_reset_vio_int`) — so either a real pin or the VIO debug probe can clear it. This gives a hardware engineer a simple way to answer “has this link glitched at all since I last checked?” without having to continuously watch `link_status_out`.

9.3 Feedback into the init FSM

`sm_link` is also fed directly into `example_init` as `rx_data_good_in` — this is the “is data actually good” signal the bring-up FSM waits for (§6.3) before declaring RX initialization truly complete, and what it continues to monitor in `ST_MONITOR` to decide whether to trigger an automatic RX-only reset/retry.

10. VIO (hardware debug) dataflow — inch by inch

A Vivado VIO IP core instance (`gtwizard_ultrascale_0_vio_0`) is wired into the free-running clock domain. It exposes:

Probe inputs (FPGA -> Vivado Hardware Manager, read-only from the GUI): - `probe_in0` = `link_status_out` - `probe_in1` = `link_down_latched_out` - `probe_in2` = `init_done_int` (from `example_init`) - `probe_in3` = `init_retry_ctr_int` (4-bit retry counter) - `probe_in4..8` = synchronized versions of `gtpowergood`, `txpmarestdone`, `rxpmarestdone`, `gtwiz_reset_tx_done`, `gtwiz_reset_rx_done` — each individually run through its own `example_bit_synchronizer` instance first, since these signals are natively asynchronous/foreign-clock-domain relative to the free-running clock and must be safely resynchronized before being sampled by the VIO's own clock-domain logic.

Probe outputs (Vivado Hardware Manager -> FPGA, writable from the GUI, for manual debug control): -

`probe_out0` -> `hb_gtwiz_reset_all_vio_int` (manually trigger a full reset) - `probe_out1` -> `hb0_gtwiz_reset_tx_pll_and_datapath_int` (manually trigger TX PLL+datapath reset) - `probe_out2` -> `hb0_gtwiz_reset_tx_datapath_int` (manually trigger TX datapath-only reset) - `probe_out3` ->

`hb_gtwiz_reset_rx_pll_and_datapath_vio_int` (manually trigger RX PLL+datapath reset — though as noted in §6.1, this path is tied off/unused for the actual reset controller input in this shared-PLL configuration) - `probe_out4` ->
`hb_gtwiz_reset_rx_datapath_vio_int` (manually trigger RX datapath-only reset) - `probe_out5` ->
`link_down_latched_reset_vio_int` (manually clear the sticky link-down flag)

This means: after programming the FPGA, an engineer opens Vivado Hardware Manager, adds this VIO's dashboard, and can watch link status/reset-done flags update live while manually forcing individual reset conditions to test bring-up robustness — all without needing to touch RTL or re-synthesize.

11. Simulation workflow (`example_top_sim.v`)

This is a **self-checking testbench**, not synthesizable, meant to be run in a simulator (XSIM/Quarta/VCS) against a behavioral (or UNIFAST-accelerated) simulation model of the transceiver core:

1. **Clock generation:**
 - `mgtrfclk0_x0y2` toggles every 8294 ps (period ≈ 16.588 ns $\approx$ matches the XDC's `create_clock -period 16.588` for the reference clock).
 - `hb_gtwiz_reset_clk_freerun` toggles every 3765 ps (period ≈ 7.529 ns, matching the XDC's free-running clock constraint and the `P_FREERUN_FREQUENCY` parameter).
2. **Loopback wiring:** `ch0_gthxn` / `ch0_gthxp` are declared as single wires connected to *both* the TX outputs and RX inputs of the DUT (`example_top`) — i.e., **direct internal loopback** at the top of the testbench, simulating a perfect cable with zero bit errors, so the design under test only has to prove its own logic (reset sequencing, PRBS generation/check, link FSM) works correctly, not real channel impairments.
3. **Reset sequencing:** `hb_gtwiz_reset_all` starts asserted, holds for 5,000,000 ps (5 μ s) plus 100 additional free-running clock edges, then deasserts — giving the reference clock and free-running clock time to be recognized as stable before releasing reset (mimicking real board power-up behavior).
4. **Pass/fail checking, in order:**
 - Wait for `hb_gtwiz_reset_all` to deassert.
 - Wait for either `link_stable` (2048 consecutive clean `link_status` cycles, tracked by its own counter) **or** a 2 ms (#2E9 ps) simulation timeout.
 - If timeout wins -> **FAIL**, "Link never achieved," `$finish`.
 - If `link_stable` wins -> print "Initial link achieved," clear the latched link-down flag (pulse `link_down_latched_reset` for 5 clock edges — proving that mechanism works too), then run for another 50 μ s (#5E7 ps) watching for any subsequent link loss.
 - At the end: **PASS** only if `link_status === 1` *and* `link_down_latched === 0` (i.e., link is currently up *and* never glitched during that 50 μ s window). Otherwise **FAIL**.

This is the standard pattern Vivado auto-generates for every GTWizd example design so you get an out-of-the-box "does my chosen configuration even work" smoke test before touching real silicon.

12. Hardware bring-up workflow (putting this on an actual board)

1. Program the bitstream containing `example_top` (built from all these files plus the core and VIO IP).
2. Apply the free-running clock and a stable differential reference clock to the constrained pins.
3. Connect a physical loopback (external SMA loopback cable, or a link partner device) to `ch0_gthtxp/n_out` and `ch0_gthrxp/n_in`.
4. Assert `hb_gtwiz_reset_all_in` briefly (or let the design free-run — the internal `example_init` FSM will self-drive reset sequencing/retries even without external intervention).
5. `example_init` walks TX reset -> RX reset -> monitors, retrying automatically (up to 15 counted attempts, tracked but not capped) if the reference clock, PLL lock, or received data isn't good within its timer windows.
6. Once PRBS31 pattern match is continuously achieved for 67 cycles, `link_status_out` goes high; `sm_link` also unblocks `example_init`'s `ST_MONITOR` state.
7. Open Vivado Hardware Manager, connect to the VIO instance, and observe/exercise the probe signals described in §10 for interactive debug — e.g., manually pulse a TX-only or RX-only reset and watch the link recover, or watch `init_retry_ctr_out` to gauge how flaky the link/clocking is.
8. `link_down_latched_out` (and its VIO mirror) gives a persistent record of any link hiccup since the last time it was cleared — useful for soak testing over time without continuously monitoring the design.

13. Summary of the complete dataflow (single diagram in words)



14. A few subtleties worth flagging explicitly

- **The RX-PLL-shared assumption:** `hb_gtwiz_reset_rx_pll_and_datapath_int` is hard-tied to `1'b0` and the comment says this configuration uses “the same PLL type for transmitter and receiver,” so this particular design cannot reset the RX PLL independently of TX — only the RX datapath. If you regenerate this example with TX/RX using different PLL types, that logic path becomes live instead of tied off.
- **`example_init`’s auto-recovery is explicitly a double-edged demo feature.** The comment in `example_top.v` calls this out directly: automatically resetting RX on link loss can fight against a user application that’s deliberately destabilizing the link (e.g., during other test/debug activity). It’s meant to be edited/removed for production use if that’s not the desired behavior.
- **Everything but the raw serial pins and two clock/reset pins is internally looped, generated, or checked** — this design does not expose a general-purpose 32-bit user data bus to the outside world; it’s purpose-built as a self-test/bring-up harness, not a data-mover. If you want to move real application data through this transceiver, you’d replace `example_stimulus_raw` / `example_checking_raw` with your own TX/RX data-path logic while keeping the reset/clocking/VIO infrastructure.
- **CDC waivers in the XDC aren’t a shortcut around real CDC risk** — they’re telling Vivado’s automatic CDC checker “this specific path is already correctly handled by an explicit synchronizer, don’t flag it as an unhandled crossing.” If you add *new* signals crossing clock domains without routing them through `example_bit_synchronizer` / `example_reset_synchronizer`, they will (correctly) get flagged.